Assignment 01 Report

Fundamentals of Natural Language Processing

Name

Muhammad Umer Shehzad

Department

Computer and Information Sciences

Batch 2023 - 2027

Computer and Information Sciences

Instructor

Sir Ijaz Hussain

Date

20th February, 2026

Part A: ELIZA-Type Mental Health Support Chatbot

1.1 How will the chatbot be used?

Mental health support is one of the most natural fits for an ELIZA-style chatbot because the original ELIZA itself was already designed as a therapist. A Rogerian approach (which involves listening, reflecting feelings, and asking open questions) maps very naturally to simple pattern matching. The chatbot does not need to give factual answers or do any reasoning. It just needs to listen and respond empathetically, which is something rule-based pattern can do reasonably well (regex).

In Pakistan especially, mental health awareness is growing but access to actual therapists is still limited. An ELIZA-type chatbot can serve as a first point of contact — a non-judgmental space where a person can express how they feel before seeking professional help.

1.2 Pronoun Reflection — The Core ELIZA Technique

This is the most important technique ELIZA uses. When the user says, "I am stressed", the bot wants to say back, "Why are you stressed?" not "Why am I stressed?". To do this, the bot swaps the pronouns automatically using a reflection table:

```
# Reflections used to guess the pronoun
REFLECTIONS = {
    "i am"    : "you are",
    "i was"   : "you were",
    "i've"    : "you have",
    "i'll"    : "you will",
```

```
    "i'd"     : "you would",
    "i'm"     : "you are",
    "i"       : "you",
    "my"      : "your",
    "me"      : "you",
    "myself"  : "yourself",
    "you are" : "I am",
    "you were": "I was",
    "you've"  : "I have",
    "you'll"  : "I will",
    "your"    : "my",
    "yours"   : "mine",
    "you"     : "me",
    "am"      : "are",
    "was"     : "were",
}
```

Two-word phrases are checked before single words to avoid partial replacements. For example, "I am" should become "you are" as a unit, not "you" + "am" separately.

1.3 Pattern Matching — How Responses are Chosen

The PATTERNS list contains tuples of (regex, responses). Each regex is tested against the user's lowercased input using re.search(). The first match wins. Here is an example pattern for anxiety:

```
# Greetings
    (r'\b(hello|hi|hey)\b|good (morning|evening|afternoon)',
     [
         "Hello! I'm here to listen and support you. How are you feeling today?",
         "Hi there. This is a safe space to share. What's on your mind?",
         "Hey! I'm glad you reached out. How are you doing today?",
     ])
```

Multiple responses per pattern are provided so the bot does not always say the exact same thing. Python's random.choice() picks one at random each time.

Patterns that use a capture group (parentheses in the regex) inject the captured text into the response via {0}. The captured text is first passed through reflect() to swap the pronouns. So if the user says "I feel very lonely", the captured group is "very lonely", which becomes "very lonely" after reflection (no pronouns here), and the response becomes "Why do you feel very lonely?"

1.4 Patterns and their responses

```
PATTERNS = [

    # Greetings
```

```
    (r'\b(hello|hi|hey)\b|good (morning|evening|afternoon)',
     [
         "Hello! I'm here to listen and support you. How are you feeling today?",
         "Hi there. This is a safe space to share. What's on your mind?",
         "Hey! I'm glad you reached out. How are you doing today?",
     ]),

    # Name introduction
    (r'my name is ([a-zA-Z]+)',
     [
         "Hello {0}! It's good to meet you. How are you feeling today?",
         "Nice to meet you, {0}. What would you like to talk about today?",
         "Welcome, {0}. I'm here to listen. How are things going for you?",
     ]),

    # CRISIS
    (r'(suicide|kill myself|end my life|want to die|don.t want to live)',
     [
         "I'm very concerned about what you just shared. Please reach out for help
right now.\n"
         "  Umang Helpline (Pakistan): 0317-4288665\n"
         "  International: www.findahelpline.com\n"
         "You are not alone, and people care about you.",

         "What you're feeling sounds very serious and your safety matters most. "
         "Please contact a crisis line immediately.\n"
         "  Umang (Pakistan): 0317-4288665\n"
         "Your life has value.",
     ]),

    (r'(self.harm|hurt myself|hurting myself|cutting myself)',
     [
         "I'm really concerned about your safety right now. Please speak to a
trusted person "
         "or contact a helpline immediately.\n"
         "  Umang (Pakistan): 0317-4288665\n"
         "You deserve care and support.",
     ]),

    # Panic / physical anxiety symptoms
    (r'(panic attack|can.t breathe|heart is racing|chest is tight|chest tight)',
     [
         "It sounds like you might be experiencing a panic attack. "
         "Let's slow down together — breathe in for 4 counts, hold for 4, out for 4.
"
```

```
        "You are safe. What is happening right now?",
        "That sounds very frightening. Try to focus on slow, steady breathing. "
        "I'm here with you. Can you tell me what triggered this?",
    ]),

    # Anxiety
    (r'(anxious|anxiety|panic|panicking|nervous|worried|worrying)',
     [
        "I'm sorry to hear you're struggling with anxiety. "
        "Can you tell me more about what's triggering these feelings?",
        "Anxiety can feel overwhelming. What thoughts are running through your mind
right now?",
        "It takes courage to acknowledge anxiety. When did these feelings start?",
        "You're not alone — many people experience this. What seems to be setting
it off?",
     ]),

    #  Depression / Sadness
    (r'i feel (sad|depressed|hopeless|empty|numb|miserable|worthless)',
     [
        "I'm sorry you're feeling {0}. Would you like to share what's been going
on?",
        "Feeling {0} can be really heavy. How long have you been feeling this
way?",
        "Thank you for trusting me with this. What do you think is contributing to
feeling {0}?",
        "You don't have to carry this alone. What has your day been like?",
     ]),

    (r'(nothing matters|pointless|no point|what.s the point|life is meaningless)',
     [
        "It sounds like you're feeling very hopeless right now. "
        "Can you tell me what's been happening that's brought you to this point?",
        "Those feelings of emptiness are very painful. Has something specific
happened recently?",
        "I hear you. When did things start feeling this way for you?",
     ]),

    # Stress / Overwhelm
    (r'(stressed|overwhelmed|too much|can.t cope|can.t handle|burned out)',
     [
        "It sounds like you're carrying a lot right now. What feels most
overwhelming?",
        "Stress can build up so quickly. What are the main things weighing on you
today?",
```

```
        "I hear you. Let's take this one step at a time. "
        "What is the single biggest source of stress for you right now?",
        "You're doing the right thing by talking about it. "
        "What would help you feel even slightly better right now?",
    ]),

    #  Loneliness
    (r'(lonely|alone|isolated|no one understands|no one cares|no friends)',
     [
        "Feeling lonely is one of the hardest experiences. "
        "What does your support network look like right now?",
        "I'm here and I'm listening. Can you tell me more about feeling alone?",
        "You reached out, which means part of you is seeking connection — "
        "that's a meaningful step. What's been making you feel isolated?",
    ]),

    # Anger
    (r'(i feel angry|i am angry|i.m angry|i.m frustrated|furious|rage|anger)',
     [
        "Anger often signals that something important to us has been hurt or
crossed. "
        "What happened?",
        "It's okay to feel angry. Can you walk me through what led to this?",
        "I understand. What do you think is underneath the anger?",
    ]),

    # Sleep problems
    (r'(can.t sleep|insomnia|not sleeping|sleep problems|awake at night|trouble
sleeping)',
     [
        "Sleep problems can make everything feel so much harder. "
        "How long have you been struggling with this?",
        "Lack of sleep affects our mood, thinking and energy a lot. "
        "What do you think is keeping you awake?",
        "That sounds exhausting. Are your thoughts racing when you try to sleep?",
    ]),

    # Family / Relationships
    (r'my (family|parents|mother|father|partner|husband|wife|friend|relationship)',
     [
        "Relationships can be really complicated. What's been happening with your
{0}?",
        "Tell me more about the situation with your {0}.",
        "How does the situation with your {0} make you feel?",
        "What do you wish your {0} understood about what you're going through?",
```

```
     ]),

    # Work / Study
    (r'(university|college|studies|exam|assignment|work|job|career|grades)',
     [
         "Academic and work pressure can be very intense. What's been happening with
{0}?",
         "Tell me more about the pressure you're feeling around {0}.",
         "How is the situation with {0} affecting your daily life and wellbeing?",
     ]),

    # Seeking professional help
    (r'(therapist|therapy|counselor|counselling|psychiatrist|psychologist)',
     [
         "Reaching out for professional support is one of the bravest steps you can
take. "
         "Have you been able to see a {0} before?",
         "That's a really positive thought. What's making you consider seeing a
{0}?",
         "A {0} can offer support that goes much deeper than what I can provide. "
         "I really encourage you to reach out to one.",
     ]),

    #  What can the bot do
    (r'(what can you do|how can you help|can you help)',
     [
         "I can listen without judgment, reflect your feelings back to you, "
         "and help you think through what's on your mind. "
         "I'm not a replacement for professional therapy, but I'm a good first step.
"
         "What would you like to talk about?",
         "I'm here to offer a safe, non-judgmental space. Tell me what's been going
on.",
     ]),

    #  Who is the bot
    (r'(who are you|what are you|are you a robot|are you human|are you real)',
     [
         "I am an ELIZA-type mental health support chatbot inspired by Joseph
Weizenbaum's "
         "original ELIZA (1966). I'm not a human or a licensed therapist, "
         "but I'm here to listen and help you reflect.",
         "I'm a chatbot designed to provide a supportive space to talk through your
feelings. "
```

```
        "For serious concerns, please reach out to a qualified mental health
professional.",
    ]),

    # Coping strategies
    (r'(what should i do|how do i cope|how can i feel better|what helps|give me
advice)',
    [
        "That's a great question. What has helped you get through difficult moments
before?",
        "Coping looks different for everyone. What small things bring you even a
little comfort?",
        "Let's think about this together. Are there activities or people that
usually "
        "help you feel more grounded?",
    ]),

    #  Feeling better / Positive
    (r'(i feel (good|better|happy|grateful|relieved|calm|hopeful))',
    [
        "I'm really glad to hear that! What's been going well?",
        "That's wonderful to hear. What do you think has helped you feel better?",
        "It's great to hear some positivity. What's bringing you this sense of
{0}?",
    ]),

    #  Thank you
    (r'\b(thank you|thanks|thank u|thx)\b',
    [
        "You're very welcome. I'm glad I could be here for you. Is there anything
else on your mind?",
        "Of course. That's what I'm here for. How are you feeling right now?",
    ]),

    #  Goodbye
    (r'\b(bye|goodbye|quit|exit|see you|take care)\b',
    [
        "Take care of yourself. Remember, it's always okay to ask for help.
Goodbye!",
        "Goodbye! I hope our conversation was helpful. "
        "Please reach out to a professional if things feel too heavy.",
        "Take good care. You matter, and so does your mental health. Goodbye!",
    ]),

    #  I don't know
```

```
    (r'(i don.t know|i.m not sure|i can.t explain|hard to explain|i don.t
understand)',
     [
         "That's perfectly okay. Take your time. "
         "Sometimes our feelings are difficult to put into words.",
         "It's alright not to have all the answers. "
         "What does it feel like, even if you can't name it exactly?",
         "Just being here and trying to talk about it is already meaningful. "
         "What's the closest you can get to describing how things feel?",
     ]),

    #  I feel ... (generic catch-all for emotions)
    (r'i feel (.*)',
     [
         "Why do you feel {0}?",
         "How long have you been feeling {0}?",
         "What do you think is making you feel {0}?",
         "Does feeling {0} affect your daily life?",
         "What would help you feel less {0}?",
         "Tell me more about feeling {0}.",
     ]),

    #  I ... (generic catch-all)
    (r'^i (.*)',
     [
         "Why do you {0}?",
         "Can you tell me more about that?",
         "How does it make you feel when you {0}?",
         "How long have you felt this way?",
         "What do you mean when you say you {0}?",
     ]),

    #  Final fallback — always matches
    (r'(.*)',
     [
         "I see. Can you tell me more about that?",
         "How does that make you feel?",
         "Let's explore that a little more. Can you say more?",
         "I'm listening. Please go on.",
         "That's important. What else is on your mind?",
         "How long have you been thinking about this?",
         "What do you think that means for you?",
     ]),
]
```

Output

```
Type 'bye' to end the session.
==================================================================

Chatbot : Hello! I'm here to listen. How are you feeling today?

You     : hello
Chatbot : Hello! I'm here to listen and support you. How are you feeling today?

You     : i am umer
Chatbot : Hello umer! It's good to meet you. How are you feeling today?

You     : bye
Chatbot : Take care of yourself. Remember, it's always okay to ask for help. Goodbye!
```

1.5 Limitations of ELIZA-Type Chatbots

- **No real understanding:** The bot matches patterns but has no idea what the words mean. It cannot reason, infer, or remember previous messages.
- **Repetitive:** If the user keeps saying things the bot does not recognise, it falls back to the same generic responses repeatedly.
- **No memory:** Each response is generated independently. The bot does not remember what was said three messages ago.
- **Cannot handle complex input:** If the user writes a long, complex sentence that does not match any pattern precisely, the bot gives a generic fallback.
- **Not a therapist:** Most importantly, ELIZA cannot replace professional mental health support. It is a first point of contact, not a diagnosis or treatment tool.

Part B: Urdu Sentence Segmentation and Tokenization

2.1 The Problem with Urdu Text

Urdu is written in Nastaliq script, right to left, using the Perso-Arabic character set. Before we can do any NLP task on Urdu text, we need to break it into meaningful units, this is called tokenization. But Urdu has two problems that make this much harder than English:

- **Space Insertion Problem:** Extra spaces get inserted inside a single word, breaking it into pieces that look like separate words but are actually one word.
- **Space Omission Problem:** Spaces between two separate words are missing, causing them to be joined together as if they were one word.

On top of these structural problems, our corpus was social media data (HS Urdu Dataset with 7,871 tweets) which added more noise — mixed English and Urdu text, hashtags, @mentions, URLs, emoji, and inconsistent punctuation. Standard off-the-shelf NLP tools are not designed for this kind of messy, informal Urdu.

2.2 Why We Built Our Own Tokenizer

Existing libraries like UrduHack are designed for formal Urdu — newspapers, books, government documents. They make assumptions about clean, properly punctuated text that simply do not hold for social media data. The assignment itself required us to build the segmentation technique from scratch, which forced us to understand the linguistic structure of Urdu rather than just calling a library function blindly.

Building from scratch also meant every decision was deliberate. We could tune the cleaning, tokenization, and segmentation rules specifically for the kind of text we actually had.

2.3 Strategy 1 — Text Cleaning (Before Tokenization)

Before any linguistic processing, we cleaned the raw text using regex patterns to remove noise. This was done in a function called clean_urdu_text():

```
def clean_urdu_text(text: str) -> str:
    if not isinstance(text, str):
        return ''
    text = text.replace('\r\n', ' ').replace('\r', ' ')
    text = re.sub(r'http\S+|www\.\S+', '', text)           # URLs
    text = re.sub(r'#\w+', '', re.sub(r'@\w+', '', text))  #hashtags / mentions
    text = re.sub(r'[a-zA-Z0-9]', '', text)                # English & digits
    text = re.sub(r'[^\u0600-\u06FF\s۔!؟]', ' ', text)  # non-Urdu chars
    return normalize_whitespace(text)
```

The Unicode range \u0600-\u06FF covers the entire Arabic/Urdu script block. Anything outside this range (except Urdu punctuation and whitespace) is replaced with a space. This elegantly handles emojis, symbols, and foreign characters all at once.

2.4 Strategy 2 — Whitespace Normalization

A separate function called normalize_whitespace() fixes both the Space Insertion and Space Omission problems:

```
def normalize_whitespace(text: str) -> str:
    text = text.replace('\u200c', ' ').replace('\u200d', '').replace('\xa0', ' ')
    text = text.replace('\r\n', ' ').replace('\r', ' ')
    text = re.sub(r'[ \t]+', ' ', text)
    text = re.sub(r' ([۔!؟])', r'\1', text)
    text = re.sub(r'([۔!])([^\s])', r'\1 \2', text)
    return text.strip()
```

\u200c is the **Zero Width Non-Joiner** and **\u200d** is the **Zero Width Joiner** , invisible characters that appear in Arabic/Urdu text and cause silent spacing issues that are impossible to spot by eye.

2.5 Strategy 3 — Tokenization

After cleaning, we tokenize the text into individual word tokens. The key insight is that Urdu punctuation marks often get attached to words without spaces, so we need to isolate them first:

```
def urdu_tokenize(text: str) -> List[str]:
    if not isinstance(text, str) or not text.strip():
        return []
    text = normalize_whitespace(text)
    text = re.sub(r'([۔؟!؛])', r' \1 ', text)
    return [t.strip() for t in text.split() if t.strip()]
```

2.6 Strategy 4 — Sentence Segmentation

Sentence segmentation means deciding where one sentence ends and the next begins. We implemented two strategies inside a class called UrduSentenceSegmenter:

Strategy 4a — Terminator-Based Splitting (Primary)

If the text contains the Urdu full stop (۔) or question mark (؟), we split on those first. This is the most reliable strategy because explicit punctuation is unambiguous:

```
def _split_on_terminators(self, text: str) -> List[str]:
        parts = re.split(r'(?<=[۔؟!])\s+', text)
        return [p.strip() for p in parts if p.strip()]
```

The regex (?<=[.?!])\s+ is a lookbehind assertion. It means: split at any whitespace that is immediately preceded by a terminator. The terminator stays with its sentence — it does not get removed.

Strategy 4b — End-Word Heuristic (Secondary)

Urdu is a verb-final language, meaning the verb almost always comes at the end of a sentence. We built two dictionaries of linguistically motivated words:

- **End-Word Dictionary (44 entries):** Verb forms that typically close a sentence — تھا, ہیں, ہے, گیا, کیا, دیا, آیا and their conjugations.
- **Start-Word Dictionary (20 entries):** Words that typically open a new sentence — پھر, لیکن, وہ, یہ, جب, کیونکہ, اس لیے etc.

A boundary is inserted only when BOTH conditions are true simultaneously — an end-word is immediately followed by a start-word AND the current sentence has at least 3 words:

```
def _endword_heuristic(self, chunk: str) -> List[str]:
        chunk = chunk.strip()
        has_terminator = chunk and chunk[-1] in URDU_TERMINATORS
        core = chunk[:-1].strip() if has_terminator else chunk

        words = core.split()
        results, current = [], []

        for i, word in enumerate(words):
            clean = remove_diacritics(word).strip('؛""\'')
            current.append(word)

            if clean in self.end_words and i + 1 < len(words):
                nxt = remove_diacritics(words[i + 1]).strip('؛""\'')
                if nxt in self.start_words and len(current) >= self.min_len:
```

```
                    results.append(' '.join(current) + '۔')
                    current = []

        # Remaining words form the last sentence
        if current:
            tail = ' '.join(current)
            # Restore original terminator on the very last piece
            tail = tail + chunk[-1] if has_terminator else tail
            results.append(tail)

        return results if results else [chunk]
```

2.7 A Key Bug We Found and Fixed

During testing with the assignment's own example sentence, we discovered that the word ھے (with ھ = U+06BE) did not match our dictionary which contained ہے (with ہ = U+06C1). These two characters look almost identical but have different Unicode code points. This is a very common issue in Pakistani social media Urdu where the Sindhi-influenced form ھ is used instead of standard ہ.

Fix 1: We added all alternate Unicode forms to the end-word dictionary — ھوا, رھا, ھیں, ھے etc.

Fix 2: We discovered the heuristic was being skipped for chunks that already ended with a terminator, meaning internal boundaries inside a sentence could never be found. We fixed this by applying the heuristic to ALL chunks regardless of whether they end with a terminator.

2.8 Evaluation

We used boundary position comparison as our evaluation metric. Each sentence list is converted into a set of cumulative word-index positions where sentences end. These sets are then compared using set operations:

Metric	Formula	Meaning
Precision	TP / (TP + FP)	Of all predicted boundaries, how many were correct
Recall	TP / (TP + FN)	Of all true boundaries, how many did we find
F1 Score	2 x P x R / (P + R)	Combined single score balancing both

Results on the HS Urdu Dataset (using 200 entries with existing terminators as gold standard): Precision 0.1347, Recall 0.0684, F1 0.0907. This is low because even the gold standard from this corpus is not rich as it contains the urdu from social media etc. Results on the UrduCorpus.txt (formal text): Precision 0.9852, Recall 1.0000, F1 0.9926.

The large difference between the two datasets makes perfect sense. Social media text uses terminators very inconsistently, making the gold standard itself noisy. Formal text is clean and well-punctuated, giving nearly perfect results. This shows our segmenter works excellently for clean Urdu but social media remains a harder challenge even for state-of-the-art tools.

2.9 Results on HS Urdu Dataset

Metric	HS Urdu Dataset (Social Media)	UrduCorpus.txt (Formal)
Total Entries	7,871	4,760
Total Tokens	268,530	25,618
Vocabulary Size	18,644 unique words	2,591 unique words
Avg Tokens per Entry	34.1	5.38
Total Sentences	17,010	4,779
F1 Score (Segmentation)	0.09 (social media noise)	0.99 (clean text)

Part C: Commentary on UrduLM Paper

Paper: "UrduLM: A Resource-Efficient Monolingual Urdu Language Model" (Naqvi et al., 2026, Habib University)

3.1 What the Paper is About — In Simple Terms

This paper from Habib University presents UrduLM — the first dedicated transformer-based language model built specifically for Urdu. Before this paper, Urdu had no monolingual language model at all. Anyone wanting to do NLP with Urdu had to use multilingual models like mBERT or XLM-RoBERTa which were trained on many languages at once and do not handle Urdu very well.

The paper makes a straightforward but important argument: a smaller model trained exclusively on Urdu data will perform better on Urdu tasks than a much larger model trained on 100+ languages where Urdu is only a tiny fraction of the data. They prove this with a 100 million parameter model that competes with multilingual models 30 times its size.

3.2 How They Built the Urdu Corpus

One of the biggest contributions of the paper is the corpus they built. Urdu had no large, publicly available clean text corpus before this work. The researchers collected 33 GB of Urdu text from four different sources:

- **Existing Datasets:** They used UrduHack and FineWeb2 as a starting point. These gave them some initial clean Urdu text to work with.
- **Web Scraping:** They crawled the internet for Urdu content from blogs, forums, news websites, poetry platforms, and Urdu Wikipedia. They were careful about domain balancing — making sure no single source dominated the corpus.
- **OCR of Books:** They scanned physical Urdu books using Google Cloud Vision OCR to digitise literary content. This is important because most classical Urdu literature only exists in physical form and has never been digitised. This adds formal, literary language to the corpus.

- **Translation:** They translated selected English text from FineWeb into Urdu using Google Translate. This was done to add structured, educational content that is not commonly found in naturally occurring Urdu web data.

After collecting all this raw text, they ran it through a cleaning pipeline that removed URLs, emails, phone numbers, English characters, and noise symbols. They also normalised the Urdu characters (handling the same Unicode confusion we encountered with ہ vs ھ), fixed word spacing errors using UrduHack's word-space mapping, and removed invisible Unicode characters.

Then they ran deduplication using MinHash-based Locality Sensitive Hashing (LSH) to remove near-duplicate content. The final corpus had 33 GB, approximately 13 million rows, and an estimated 5-6 billion tokens. This is, according to the authors, the largest curated Urdu dataset ever built.

3.3 How UrduLM is Different from mBERT and XLM-RoBERTa

The key difference is focus. mBERT (Multilingual BERT) was trained on 104 languages at once. XLM-RoBERTa was trained on 100 languages. Both give Urdu only a very small fraction of their training data and vocabulary. UrduLM was trained exclusively on Urdu.

This matters for several concrete reasons:

Vocabulary Allocation

mBERT has a vocabulary of about 110,000 tokens shared across 104 languages. Urdu gets only a small slice of this. XLM-RoBERTa has 250,000 tokens for 100 languages. In contrast, UrduLM-32k has 32,000 tokens dedicated entirely to Urdu. This means the vocabulary is much more meaningful for Urdu — every slot is used for an Urdu word or subword, not shared with Arabic, Persian, Hindi, or any other language.

Capacity Allocation

The paper makes an excellent analogy. A 3 billion parameter multilingual model supporting 100 languages effectively allocates only about 30 million parameters to Urdu (assuming uniform distribution — in practice even less because high-resource languages dominate). UrduLM's 100 million parameters are all dedicated to Urdu. So despite being 30 times smaller, it actually has more Urdu-specific capacity.

Cultural and Linguistic Knowledge

Multilingual models often internally translate Urdu reasoning into English because most of their training was in English. A paper cited by UrduLM's authors (Wendler et al., 2024) found that LLaMA-2 does 89.7% of its computation in English regardless of the input language. This means it loses Urdu-specific cultural references, idioms, and context. UrduLM avoids this entirely.

Feature	mBERT	XLM-RoBERTa	UrduLM-32k
Languages	104	100	1 (Urdu only)
Parameters	110M	270M-560M	100-134M
Vocabulary	110k (shared)	250k (shared)	32k (Urdu only)
Tokenizer	WordPiece (multilingual)	SentencePiece (BPE)	Custom BPE (Urdu)
Architecture	Encoder-only	Encoder-only	Decoder-only (GPT)
Script Handling	Generic Unicode	Generic Unicode	Urdu Unicode ranges

Feature	mBERT	XLM-RoBERTa	UrduLM-32k
Training Cost	Extremely high	Extremely high	< $100

3.4 How UrduLM's Tokenization Differs from What We Studied in Class

In our FNLP class we studied several tokenization strategies. Here is how UrduLM's approach relates to and differs from each:

Whitespace Tokenization (What We Built in Our Assignment)

The simplest approach — split on spaces. Works fine for clean English but terrible for Urdu because of the Space Insertion and Space Omission problems we discussed in Part B. It also cannot handle morphologically complex words at all.

WordPiece (Used by BERT and mBERT)

WordPiece splits words into subword units based on frequency. It was designed for English and Latin scripts. When applied to Urdu, it does not recognise Urdu characters well, produces many single-character tokens, and splits words in linguistically meaningless ways. For example, it might split an Urdu verb form into pieces that cross morpheme boundaries, destroying grammatical meaning.

SentencePiece BPE (Used by XLM-RoBERTa)

SentencePiece with BPE (Byte Pair Encoding) is language-agnostic and works without pre-tokenization. It is better than WordPiece for non-Latin scripts but still has a problem: when trained on 100 languages, the vocabulary gets dominated by high-resource languages and Urdu gets very few subword units. This means Urdu words get split into many small fragments, causing high fertility (the number of tokens per word).

UrduLM's Custom BPE Tokenizer

The UrduLM tokenizer uses BPE (the same algorithm as XLM-RoBERTa) but with three critical differences:

- **Trained only on Urdu:** All 32,000 vocabulary slots are filled with Urdu-specific subwords that reflect actual Urdu morphology.
- **Script-aware regex:** The authors adapted OpenAI's GPT-4 regex patterns to handle Urdu Unicode ranges (U+0600 to U+06FF), Urdu numerals, and Urdu punctuation specifically. Generic regex patterns used by mBERT and XLM-RoBERTa are designed for Latin scripts and fail on Perso-Arabic characters.
- **Morphological alignment:** Because the BPE was trained on Urdu text, the merge operations naturally learn Urdu-specific patterns — suffixes, verb endings, and grammatical markers that are meaningful in Urdu.

3.6 Applied Example — Hate Speech Detection on HS Urdu Dataset

The HS Urdu Dataset we used throughout this assignment is a hate speech classification dataset with 7,871 tweets labelled as hate (h), offensive (o), or neutral (n). This is a very relevant benchmark for comparing tokenization approaches because:

- **Hate speech often uses non-standard spelling:** Offensive words frequently appear with alternate Unicode forms, extra characters, or deliberate misspellings to avoid detection. A tokenizer trained specifically on Urdu is more robust to these variations.

- **Social media Urdu is informal:** It uses regional dialectal forms, code-mixing, and abbreviations that multilingual tokenizers handle poorly because their training data was mostly formal text.
- **Sentiment and cultural context matter:** The paper's own evaluation showed UrduLM outperforms even larger multilingual models on sentiment classification (66.6% vs Qwen 1.5B's 50.2%) specifically because it captures idiomatic and culturally-specific patterns that multilingual models miss.

For hate speech detection specifically, where understanding Urdu-specific offensive terminology, cultural insults, and regional expressions is critical, the UrduLM tokenizer would provide a significant advantage over mBERT or XLM-RoBERTa. Words that multilingual tokenizers fragment into meaningless subword pieces would be preserved as meaningful units in UrduLM's vocabulary.

3.7 Which Model is Better for Urdu Tasks?

Based on the paper's findings and our analysis, the answer depends on the task and the resources available:

UrduLM is Better For:

- **Sentiment and hate speech classification:** Language-specific pretraining captures cultural idioms, regional expressions, and offensive terminology that multilingual models miss. The paper proves this with 66.6% accuracy vs Qwen 1.5B's 50.2%.
- **Open-domain question answering:** Where Urdu-specific world knowledge (Pakistani history, culture, geography) is needed, UrduLM's monolingual corpus is more comprehensive.
- **Low-resource deployment in Pakistan:** Training cost under $100 and ~32 kg CO2 makes it feasible for Pakistani universities and research institutions without access to expensive compute.
- **Real-time applications:** 2.75 times fewer tokens mean faster inference, lower latency, and lower cost per inference call.

Multilingual Models are Better For:

- **Cross-lingual tasks:** If the task involves comparison between Urdu and English, or translation, a multilingual model that knows both languages is necessary.
- **When very little Urdu data is available:** For fine-tuning scenarios with under 1-2 GB of task-specific data, XLM-RoBERTa's broad multilingual knowledge provides useful transfer.
- **Complex tasks requiring world knowledge:** Grammar error correction (BLEU 30.59 vs 35-40) and extractive QA (ROUGE-L 0.17 vs 0.35-0.47) still favor larger multilingual models because these tasks benefit from the sheer scale of parameters.

3.8 Limitations of UrduLM Acknowledged in the Paper

The authors are honest about what their model cannot do well yet:

- **Only 100M parameters:** Complex tasks like grammar correction and extractive QA still require more capacity than 100M provides.
- **Context length of only 1024 tokens:** Multilingual models like Qwen 2.5 support up to 128k tokens. UrduLM cannot handle long documents well.
- **No instruction tuning yet:** The model is a base pretrained model, not a chat model. It cannot follow instructions the way GPT-4 or LLaMA-chat can.

- **Dialectal gaps:** The corpus is mostly standard Pakistani Urdu. Regional dialects, Punjabi-Urdu code-mixing, and Sindhi-influenced Urdu are underrepresented.
- **Evaluation benchmark quality:** Urdu lacks expert-annotated gold standard benchmarks. The evaluation datasets used were adapted from English, which may not capture authentic Urdu language use.

3.9 Overall Conclusion

This paper (Urdu_LM) reinforces the things we have learned in a specific language and its rules. It also tells that we do not need billions of parameters; the 100M parameter language specific model is performing better than the billion-parameter multilingual model. With proper corpus, it's cleaning and tokenization we can train our model better. This is demonstrated by the HS Urdu Dataset which consists of hate speech, social media language that does is not properly structured making it excellent for training the model because it would require different rules for tokenization, thus capturing the irregular patterns in Urdu too.

End of Report

NLP Assignment — Fundamentals of Natural Language Processing